

Programming of Reliable Embedded Systems

Lab 3: Reliability Analysis and TMR Design

Spring 2026

1 Introduction and Objectives

This lab investigates reliability in embedded digital systems through fault injection, inequality monitoring, and Triple Modular Redundancy (TMR). You will design modular VHDL components, integrate them into fault-prone ALU architectures, and evaluate reliability behaviour through simulation.

Learning Objectives

By completing this project, students will:

- Implement modular digital components in VHDL.
- Understand the effect of injected faults on system behaviour.
- Analyse system reliability using inequality monitoring.
- Apply Triple Modular Redundancy (TMR) for fault tolerance.
- Interpret long-run simulation results using graphical analysis.

2 Project Overview

The project is divided into seven structured implementation steps, followed by simulation analysis and reporting.

All modules must be individually tested using dedicated testbenches and then integrated into higher-level reliability architectures.

3 Step-by-Step Design Tasks

Step 1 — Counter Module

Implement a synchronous counter with:

- Inputs: `clk`, `reset`, `en`
- Output: integer `counter_out`

Functional behaviour:

- Reset initializes the output to zero on the **falling clock edge**.
- When reset is inactive and `en = 1`, **the counter increments on the falling edge**.
- Note: This is the only module triggered on the falling edge; all others use rising-edge clocking.

A dedicated testbench must verify correct operation.

Step 2 — Comparator Module

Design a **generic n-bit comparator**:

- Inputs: `cmp_in1`, `cmp_in2`
- Output: `cmp_out`

Behaviour:

- Output is 1 when inputs **are not equal**
- Output is 0 otherwise.

Test with $n = 4$

Step 3 — Inequality Counter

Create a generic n -bit inequality counter by integrating:

- The previously designed Counter
- The Comparator module

```

1 entity Inequality_Counter is
2   generic (n : natural);
3   port (
4       in1,in2 : in std_logic_vector(n-1 downto 0);
5       clk,reset: in std_logic;
6       ineq_counter_out: out integer );
7 end Inequality_Counter;

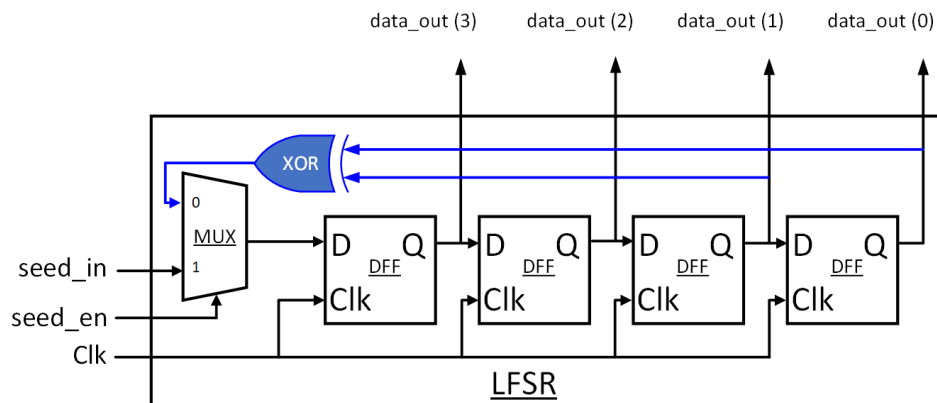
```

Test using $n = 4$.

Step 4 — LFSR Module

Implement a generic n -bit **Linear Feedback Shift Register (LFSR)** using only **FOR-GENERATE** statements, instantiations, and concurrent assignments.

- An example 4-bit LFSR is shown in the provided figure.
- The LFSR is used to generate a long pseudo-random sequence of test cases.
- Seeding procedure:
 - Set $\text{seed_en} = 1$ to enable seeding and feed a sequence of seed_in values (typically n values for an n -bit LFSR).
 - After at least n clock cycles, set $\text{seed_en} = 0$ and keep it low permanently. The output sequence is then generated.
 - The seed value must not be all zeros.
 - Connect bits 0 and 1 to the XOR feedback, independent of n .
- Inputs: 1-bit seed_en , seed_in , clk
- Output: n -bit data_out
- Verify the design using a dedicated testbench (use an 8-bit LFSR).



Step 4b — Fault Location Generator (FLG)

You will use the provided (on last page) **random fault-location generator** that:

- generates random fault locations.
- The `fault_probability` parameter determines the fraction of the times that the fault location is between 0 and 10. For example, when you set this parameter to 0.2, in (almost) 20% of the times, a random value between 0 and 10 is generated, and in (almost) 80% of the times, the value 15 ("1111") is generated.
- Note that this parameter is of real type and a real value should be assigned to it. So, even if you want to assign the value of "1" to it (to get the fault location 100% of the times), you should set this value to "1.0".
- If you instantiate multiple instances of an FLG in a single architecture, you need to set different seeds for them.
- Note that this code uses some VHDL statements that make it impossible to be opened in the Elaborated Design view (RTL Analysis) in Vivado.

Step 5 — Faulty ALU Tester

Construct the architecture shown in the project diagram:

- LFSR and FLG operate on the **rising clock edge**.
- LFSR generates 11 bits for ALU inputs and opcode.
- Declare a generic for FLG fault probability (set to 0.2 in the testbench).
- Simulate for **100,000 clock cycles**.

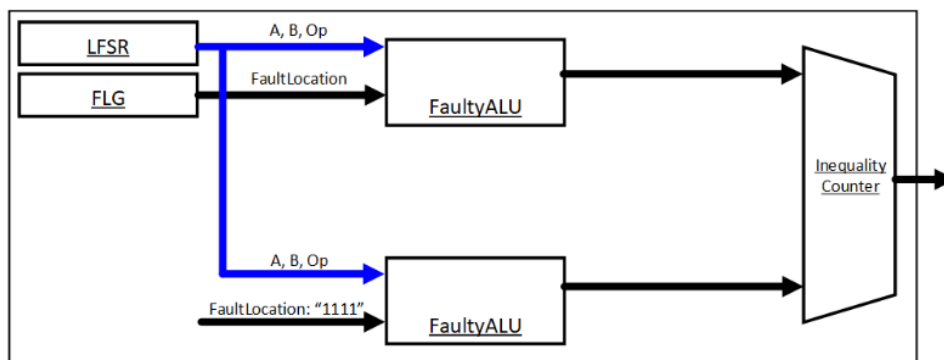


Figure 1: The diagram illustrates the interconnection between LFSR, FLG, Faulty ALUs, and the inequality counter.

Step 5b — Faulty ALU (Provided)

The Faulty ALU is built from two blocks: **ArithLogic** and **FaultInjector**.

- **ArithLogic**: takes two 4-bit inputs `A`, `B` and produces eight 4-bit results: `AplusB`, `AandB`, `AxorB`, `AnandB`, `AplusOne`, `Ao`, `Bo`, and `AllZero`. For `AplusB` and `AplusOne`, discard any carry-out (keep only the lower 4 bits).
- **FaultInjector**: takes `A`, `B`, `Operation[2:0]`, and `FaultLocation[3:0]`. If `FaultLocation` is 0–10 (binary "0000" – "1010"), it flips exactly one bit: 0–3 flips `A(0) – A(3)`, 4–7 flips `B(0) – B(3)`, and 8–10 flips `Operation(0) – Operation(2)`. For `FaultLocation` ≥ 11 , no bit is flipped.

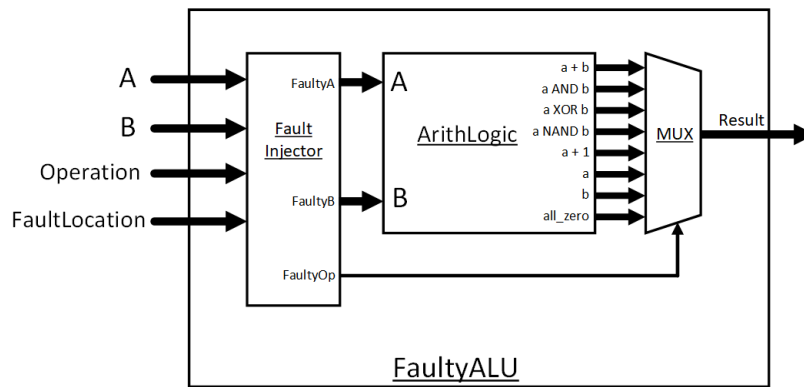
Step 6 — Triple Modular Redundancy (TMR)

Implement a generic **n-bit TMR voter**:

- Inputs: `tmr_in1`, `tmr_in2`, `tmr_in3`
- Output: `tmr_out`

Voting rule:

- Output equals **the majority value**.



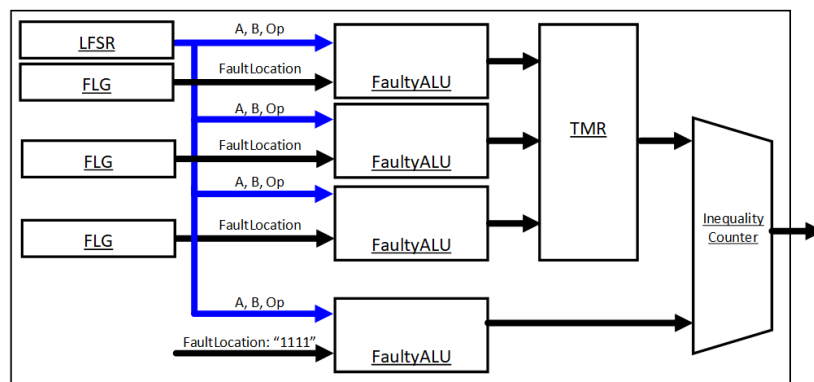
- If no majority exists, output equals **the first input**.

Step 7 — Faulty ALU Tester with TMR

Extend the previous architecture by inserting **the TMR voter**.

Requirements:

- Rising-edge triggering for LFSR and FLGs.
- Generic FLG fault probability.
- Simulate **100,000 cycles** under three conditions:
 - Fault probability = 0.2
 - Fault probability = 0.5
 - Fault probability = 1.0



4 Simulation Analysis and Reporting

The final report must include:

- A line graph of:
 - Clock cycles vs. inequality counter output
 - Samples approximately every ****20,000 cycles****
- A **combined comparison graph** including:
 - Step 5 design
 - Step 7 TMR design
- Calculation of simulation duration required for 100,000 clock periods (in ps or ns).

Fault Location Generator (FLG)

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.math_real.all;
4  use ieee.numeric_std.all;
5
6  entity fault_location_generator is
7      generic (fault_probability : real; seed: positive);
8      port (
9          clk          : in  std_logic;
10         fault_location : out std_logic_vector (3 downto 0)
11     );
12 end;
13
14 architecture arch of fault_location_generator is
15 begin
16     process (clk)
17         variable seed1, seed2 : positive;
18         variable rand : real;
19     begin
20         if (clk = '1') then
21             seed1 := seed1 + seed;
22             seed2 := seed2 + 2 * seed + 1;
23             uniform(seed1, seed2, rand);
24             if (rand < fault_probability) then
25                 seed1 := seed1 + seed;
26                 seed2 := seed2 + 2 * seed + 1;
27                 uniform(seed1, seed2, rand);
28                 fault_location <= std_logic_vector(to_unsigned(integer(floor(rand * real(1))), 4));
29             else
30                 fault_location <= "1111";
31             end if;
32         end if;
33     end process;
34 end;
35
36 library ieee;
37 use ieee.std_logic_1164.all;
38 use ieee.math_real.all;
39
40 entity tb is end;
41 architecture tb_arch of tb is
42     component fault_location_generator is
43         generic (fault_probability : real; seed: positive);
44         port (
45             clk          : in  std_logic;
46             fault_location : out std_logic_vector (3 downto 0)
47         );
48     end component;
49
50     signal clk : std_logic := '0';
51     signal fault_location1 : std_logic_vector (3 downto 0);
52     signal fault_location2 : std_logic_vector (3 downto 0);
53     signal fault_location3 : std_logic_vector (3 downto 0);
54 begin
55     clk <= not clk after 10 ps;
56     inst1 : fault_location_generator generic map (0.5, 1) port map (clk, fault_location1);
57     inst2 : fault_location_generator generic map (0.5, 2) port map (clk, fault_location2);
58     inst3 : fault_location_generator generic map (0.5, 3) port map (clk, fault_location3);
59 end;
```

FaultyALU code

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  -- 1. Fault Injector Block
6  entity FaultInjector is
7      port(
8          A, B          : in  std_logic_vector(3 downto 0);
9          Operation      : in  std_logic_vector(2 downto 0);
10         FaultLocation : in  std_logic_vector(3 downto 0);
11         FaultyA        : out std_logic_vector(3 downto 0);
12         FaultyB        : out std_logic_vector(3 downto 0);
13         FaultyOp       : out std_logic_vector(2 downto 0)
14     );
15 end FaultInjector;
16
17 architecture Behavioral of FaultInjector is
18 begin
19     -- Inject faults into A based on location 0000-0011
20     with FaultLocation select
21         FaultyA <= (A xor "0001") when "0000",
22                    (A xor "0010") when "0001",
23                    (A xor "0100") when "0010",
24                    (A xor "1000") when "0011",
25                    A              when others;
26
27     -- Inject faults into B based on location 0100-0111
28     with FaultLocation select
29         FaultyB <= (B xor "0001") when "0100",
30                    (B xor "0010") when "0101",
31                    (B xor "0100") when "0110",
32                    (B xor "1000") when "0111",
33                    B              when others;
34
35     -- Inject faults into the Operation code
36     with FaultLocation select
37         FaultyOp <= Operation xor "001" when "1000",
38                    Operation xor "010" when "1001",
39                    Operation xor "100"  when "1010",
40                    Operation              when others;
41 end Behavioral;
```

```

1  -- 2. Arithmetic and Logic Block
2  library ieee;
3  use ieee.std_logic_1164.all;
4  use ieee.numeric_std.all;
5
6  entity ArithLogic is
7      port(
8          A, B      : in  std_logic_vector(3 downto 0);
9          FaultyOp  : in  std_logic_vector(2 downto 0);
10         Result    : out std_logic_vector(3 downto 0)
11     );
12 end ArithLogic;
13
14 architecture Behavioral of ArithLogic is
15 begin
16     -- Combined ArithLogic and Mux functionality
17     process(A, B, FaultyOp)
18     begin
19         case FaultyOp is
20             when "000" => Result <= std_logic_vector(unsigned(A) + unsigned(B)); -- A + B
21             when "001" => Result <= A and B; -- A AND B
22             when "010" => Result <= A xor B; -- A XOR B
23             when "011" => Result <= A nand B; -- A NAND B
24             when "100" => Result <= std_logic_vector(unsigned(A) + 1); -- A + 1
25             when "101" => Result <= A; -- Pass A
26             when "110" => Result <= B; -- Pass B
27             when "111" => Result <= "0000"; -- All Zero
28             when others => Result <= "0000";
29         end case;
30     end process;
31 end Behavioral;
32 -----
33 -- 3. Top Level FaultyALU
34 -----
35 library ieee;
36 use ieee.std_logic_1164.all;
37
38 entity FaultyALU is
39     port(
40         A, B      : in  std_logic_vector(3 downto 0);
41         Operation  : in  std_logic_vector(2 downto 0);
42         FaultLocation : in std_logic_vector(3 downto 0);
43         Result      : out std_logic_vector(3 downto 0)
44     );
45 end FaultyALU;
46
47 architecture Structural of FaultyALU is
48     signal TFaultyA, TFaultyB : std_logic_vector(3 downto 0);
49     signal TFaultyOp          : std_logic_vector(2 downto 0);
50 begin
51     -- Instantiating the two main blocks
52     Injector: entity work.FaultInjector(Behavioral)
53         port map (A, B, Operation, FaultLocation, TFaultyA, TFaultyB, TFaultyOp);
54
55     ALU_Core: entity work.ArithLogic(Behavioral)
56         port map (TFaultyA, TFaultyB, TFaultyOp, Result);
57 end Structural;

```

```

1  -----
2  -- 4. Testbench for FaultyALU
3  -----
4  library ieee;
5  use ieee.std_logic_1164.all;
6
7  entity FaultyALU_tb is
8  end FaultyALU_tb;
9
10 architecture tb of FaultyALU_tb is
11     signal A, B          : std_logic_vector(3 downto 0) := "0000";
12     signal Operation      : std_logic_vector(2 downto 0) := "000";
13     signal FaultLocation  : std_logic_vector(3 downto 0) := "1111"; -- Default No Fault
14     signal Result         : std_logic_vector(3 downto 0);
15 begin
16     -- Unit Under Test
17     UUT: entity work.FaultyALU(Structural)
18         port map (A, B, Operation, FaultLocation, Result);
19
20     -- Stimulus process
21     process
22     begin
23         -- Test Case 1: Simple Addition (No Fault)
24         A <= "0010"; B <= "0011"; Operation <= "000"; FaultLocation <= "1111";
25         wait for 10 ns;
26
27         -- Test Case 2: Addition with Fault in A (Bit 0 flipped)
28         FaultLocation <= "0000";
29         wait for 10 ns;
30
31         -- Test Case 3: XOR operation with Fault in Op code
32         A <= "1010"; B <= "1100"; Operation <= "010"; FaultLocation <= "1000";
33         wait for 10 ns;
34
35         -- End Simulation
36         wait;
37     end process;
38 end tb;

```